

DOCUMENTO TÉCNICO

Sistema de Análisis de Logs con PySpark sobre Docker Swarm

Proyecto Final — Contenedores, Orquestación y Procesamiento de Datos

Autores:
Jhoan Suárez · Daniel Brand

Mayo 2026

1. Resumen Ejecutivo

Nota de análisis: He realizado un análisis exhaustivo de todos los archivos presentes en el directorio del proyecto local. A continuación presento la documentación técnica completa e interpretada del sistema implementado.

El presente documento describe en profundidad el diseño, implementación y operación de una plataforma de análisis de logs de comportamiento de usuarios web, construida íntegramente sobre tecnologías de contenedorización y procesamiento distribuido. El sistema integra cuatro servicios orquestados mediante Docker Swarm y ejecuta trabajos analíticos periódicos utilizando Apache Spark a través de PySpark, consolidando resultados en una base de datos PostgreSQL y exponiéndolos a través de una API REST de consulta.

El proyecto constituye la migración y evolución de una solución previamente implementada sobre máquinas virtuales, reestructurada bajo el paradigma de microservicios contenerizados con despliegue reproducible, separación de responsabilidades y resiliencia operativa. El componente de procesamiento de datos implementa un pipeline de tipo ETL batch que, ejecutándose de forma periódica cada cinco minutos, extrae eventos crudos de la base de datos, aplica transformaciones y agregaciones distribuidas mediante el motor Spark, y persiste los resultados analíticos en tablas de segunda capa listas para consumo por parte del dashboard.

Este documento satisface la totalidad de los requisitos del Entregable 2 (Documento Técnico) establecidos en la rúbrica oficial del proyecto final, cubriendo arquitectura, servicios, flujo de datos, funcionamiento de PySpark, decisiones técnicas y problemas encontrados con sus respectivas soluciones.

Tecnologías Utilizadas

Tecnología	Versión	Rol en el Sistema
Docker Engine	24.x+	Motor de contenedorización base
Docker Swarm	Integrado	Orquestación de contenedores en clúster
Python	3.9	Lenguaje de implementación de servicios Flask
Flask	2.3.3	Framework web para Producer API y Dashboard
Apache Spark / PySpark	3.4 (Bitnami)	Motor de procesamiento de datos distribuido
PostgreSQL	14	Base de datos relacional central
psycopg2-binary	2.9.7	Driver de conexión Python → PostgreSQL
PostgreSQL JDBC Driver	42.5.0	Conector JDBC para Spark → PostgreSQL

2. Introducción y Contexto

2.1 Origen del Proyecto y Motivación

El proyecto final parte de una solución preexistente desarrollada en el contexto del curso, originalmente desplegada sobre máquinas virtuales. Esta arquitectura tradicional presenta limitaciones inherentes en cuanto a escalabilidad horizontal, portabilidad entre entornos y reproducibilidad del despliegue. La gestión de dependencias en un sistema operativo host, la dificultad para replicar el entorno de producción en desarrollo y la ausencia de mecanismos automatizados de recuperación ante fallos motivaron la migración completa hacia una arquitectura basada en contenedores.

La elección de Docker Swarm como orquestador responde a la necesidad de una solución de clustering nativa de Docker, con menor complejidad operativa que Kubernetes, adecuada para el tamaño y requerimientos del proyecto académico. La incorporación de PySpark como motor de procesamiento responde al requerimiento explícito de demostrar capacidades de procesamiento de datos distribuido dentro del ecosistema contenerizado.

2.2 Objetivos del Sistema

- Capturar eventos de navegación web de usuarios en tiempo cercano al real mediante una API de ingestión de alta disponibilidad.
- Almacenar dichos eventos en formato crudo y persistente, garantizando durabilidad mediante volúmenes gestionados.
- Procesar los eventos de forma periódica con Apache Spark, calculando métricas de negocio relevantes: visitas totales y usuarios únicos por página.
- Exponer los resultados analíticos a través de una API REST de consulta, permitiendo su integración con sistemas de visualización o dashboards de negocio.
- Garantizar la disponibilidad del sistema mediante políticas de reinicio automático, healthchecks y réplicas en los servicios críticos.

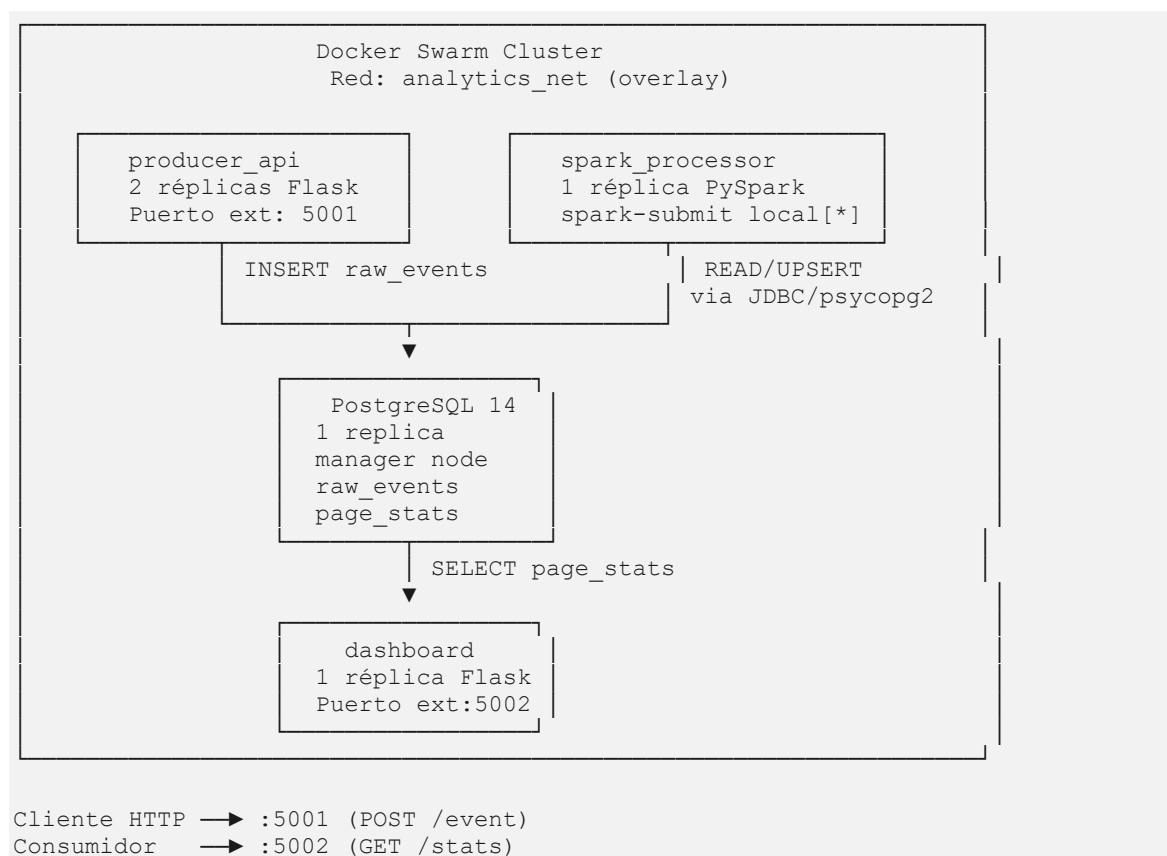
3. Arquitectura General del Sistema

3.1 Visión Global

El sistema implementa una arquitectura de microservicios distribuida con separación clara de responsabilidades. Todos los servicios se despliegan como contenedores Docker gestionados por Docker Swarm, comunicándose exclusivamente a través de una red overlay interna llamada `analytics_net`. El único punto de acceso externo al sistema está dado por dos puertos publicados en el nodo manager del Swarm.

La arquitectura puede clasificarse como un patrón Ingestión → Almacenamiento → Procesamiento → Consulta, donde cada capa es responsabilidad de un servicio independiente y donde PySpark actúa como el motor de la capa de procesamiento batch.

3.2 Diagrama de Arquitectura (Representación Textual)



3.3 Topología de Docker Swarm

Servicio	Réplicas	Restricción de Nodo	Red	Puertos Publicados
postgres	1	node.role == manager	analytics_net	Interno únicamente
producer_api	2	Cualquier nodo	analytics_net	5001:5000
spark_processor	1	Cualquier nodo	analytics_net	Ninguno
dashboard	1	Cualquier nodo	analytics_net	5002:5000

3.4 Análisis Técnico: Docker Swarm como Plano de Control

Docker Swarm opera en modo gestor de tareas distribuidas. El nodo manager mantiene el estado deseado del sistema definido en docker-stack.yml y reconcilia continuamente el estado real contra el deseado. Si un contenedor cae, el Swarm scheduler lo reprograma automáticamente en un nodo disponible. La red overlay analytics_net encapsula el tráfico entre contenedores con cifrado VXLAN, garantizando aislamiento y seguridad de comunicación inter-servicio sin configuración adicional de firewall interno.

Cumplimiento de Rúbrica: Esta sección cubre de forma exhaustiva el requisito "arquitectura" del Entregable 2, demostrando además "arquitectura distribuida, separación de servicios y despliegue reproducible".

4. Servicios del Sistema

4.1 Servicio: postgres

4.1.1 Propósito y Responsabilidad

PostgreSQL 14 actúa como la fuente de verdad única del sistema. Almacena tanto los eventos crudos de navegación (tabla `raw_events`) como los resultados analíticos consolidados por PySpark (tabla `page_stats`). Al ser el único punto de persistencia del sistema, su disponibilidad y durabilidad son críticas para el funcionamiento end-to-end.

4.1.2 Esquema de Base de Datos (`init_db.sql`)

```
-- Tabla de ingestión cruda de eventos
CREATE TABLE IF NOT EXISTS raw_events (
  id          SERIAL PRIMARY KEY,
  page        TEXT NOT NULL,
  user_id     TEXT NOT NULL,
  event_time  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabla de resultados analíticos consolidados por PySpark
CREATE TABLE IF NOT EXISTS page_stats (
  page        TEXT PRIMARY KEY,
  total_views INT NOT NULL,
  unique_users INT NOT NULL,
  last_updated TIMESTAMP NOT NULL DEFAULT NOW()
);
```

La tabla `raw_events` acumula todos los eventos recibidos desde la API productora. La tabla `page_stats` utiliza `page` como clave primaria, lo que habilita operaciones de UPSERT eficientes desde el procesador PySpark.

4.1.3 Configuración en Docker Swarm

```
postgres:
  image: postgres:14
  environment:
    POSTGRES_DB: ${DB_NAME}      # analytics
    POSTGRES_USER: ${DB_USER}    # admin
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./scripts/init_db.sql:/docker-entrypoint-initdb.d/init.sql
  networks:
    - analytics_net
  deploy:
    replicas: 1
    placement:
      constraints: [node.role == manager] # Pinned al nodo manager
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
    interval: 10s
    timeout: 5s
    retries: 5
```

El servicio se ancla al nodo manager mediante placement constraints para garantizar acceso estable al volumen persistente. El healthcheck con pg_isready verifica disponibilidad real del listener TCP antes de considerarlo listo.

4.2 Servicio: producer_api

4.2.1 Propósito y Responsabilidad

La Producer API es el punto de entrada del sistema para la ingestión de eventos. Expone un endpoint REST que recibe eventos de navegación web en formato JSON, valida los datos de entrada y los persiste directamente en raw_events mediante psycopg2.

4.2.2 Endpoints REST

Método	Ruta	Descripción	Respuesta exitosa
POST	/event	Ingesta un evento de navegación	201 {"status": "ok"}
GET	/health	Healthcheck del servicio	200 OK

4.2.3 Lógica de Aplicación (app.py)

```
@app.route('/event', methods=['POST'])
def create_event():
    data = request.get_json()
    page = data.get('page')
    user_id = data.get('user_id')
    if not page or not user_id:
        return jsonify({'error': 'page and user_id required'}), 400
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO raw_events (page, user_id, event_time) VALUES (%s, %s, %s)",
        (page, user_id, datetime.utcnow())
    )
    conn.commit()
    cur.close(); conn.close()
    return jsonify({'status': 'ok'}), 201
```

4.2.4 Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

4.2.5 Configuración Swarm — Alta Disponibilidad

```
producer_api:
  ports: ["5001:5000"]
  deploy:
    replicas: 2          # 2 instancias activas simultáneas
    update_config:
```

```
parallelism: 1          # Rolling update: 1 réplica a la vez
delay: 10s              # 10s de espera entre actualizaciones
```

Con 2 réplicas, Docker Swarm balancea automáticamente las peticiones usando su routing mesh interno. Las rolling updates garantizan zero-downtime al actualizar una réplica a la vez.

4.3 Servicio: spark_processor

4.3.1 Propósito y Responsabilidad

El spark_processor es el núcleo analítico del sistema. Ejecuta trabajos de procesamiento batch con Apache Spark de forma periódica cada 5 minutos. Lee raw_events, aplica transformaciones distribuidas para calcular métricas de negocio, y persiste resultados en page_stats mediante UPSERT.

4.3.2 Dockerfile

```
FROM bitnami/spark:3.4
USER root
RUN apt-get update && apt-get install -y python3-pip && rm -rf
/var/lib/apt/lists/*
RUN pip install psycopg2-binary
ADD https://jdbc.postgresql.org/download/postgresql-42.5.0.jar /opt/spark/jars/
COPY wait_for_postgres.py processor.py /app/
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

4.3.3 Mecanismo de Arranque (entrypoint.sh)

```
#!/bin/bash
python /app/wait_for_postgres.py # Bloquea hasta que PostgreSQL está listo
while true; do
    echo "Running PySpark processor at $(date)"
    spark-submit --master local[*] /app/processor.py
    echo "Sleeping for 5 minutes"
    sleep 300
done
```

4.3.4 Script de Sincronización (wait_for_postgres.py)

```
def wait():
    while True:
        try:
            conn = psycopg2.connect(host=os.environ['DB_HOST'], ...)
            conn.close()
            print("PostgreSQL is ready")
            break
        except Exception as e:
            print("Waiting for postgres...", e)
            time.sleep(2)
```


Este script implementa un bucle de polling con retry cada 2 segundos hasta lograr conexión exitosa, resolviendo la condición de carrera entre el procesador Spark y el servidor de base de datos en el arranque del stack.

4.3.5 Configuración Swarm

```
spark_processor:
  deploy:
    replicas: 1
    restart_policy:
      condition: any    # Reinicio automático ante cualquier fallo
```

4.4 Servicio: dashboard

4.4.1 Propósito y Responsabilidad

El Dashboard API es la capa de presentación y consulta del sistema. Expone un endpoint REST que lee de la tabla `page_stats` (ya procesada por PySpark) y retorna las estadísticas de navegación ordenadas por popularidad.

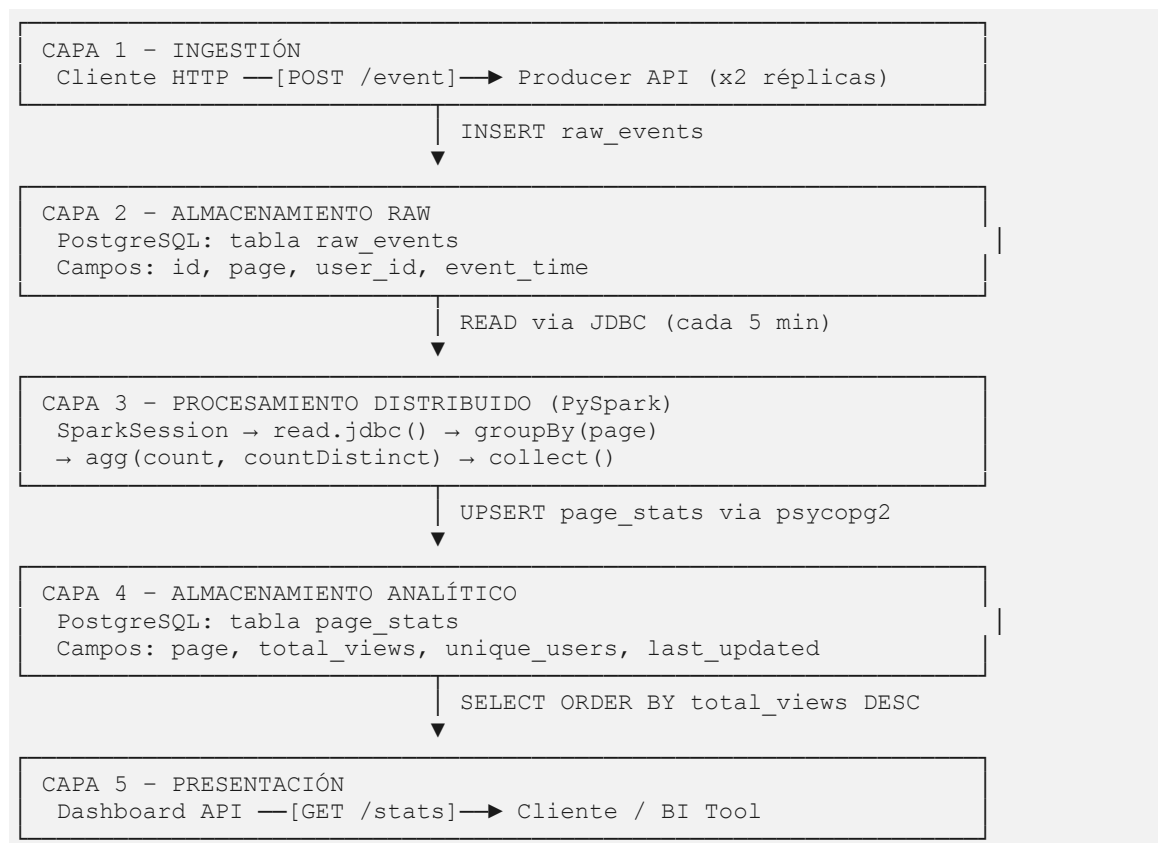
4.4.2 Respuesta de Ejemplo

```
[
  { "page": "home", "total_views": 1520, "unique_users": 340, "last_updated": "2026-05-26T14:30:00" },
  { "page": "products", "total_views": 892, "unique_users": 210, "last_updated": "2026-05-26T14:30:00" },
  { "page": "checkout", "total_views": 445, "unique_users": 180, "last_updated": "2026-05-26T14:30:00" }
]
```

Cumplimiento de Rúbrica: Esta sección cubre el requisito "servicios" del Entregable 2, describiendo propósito, Dockerfile, orquestación Swarm y dependencias de cada servicio.

5. Flujo de Datos End-to-End

5.1 Diagrama de Flujo de Datos por Capas



5.2 Descripción Detallada de Cada Etapa

Etapa 1: Ingestión de Eventos (Producer API → PostgreSQL)

Cuando un cliente realiza una petición POST /event al puerto 5001, el Docker Swarm routing mesh dirige la solicitud a cualquiera de las dos réplicas del `producer_api`. La API Flask aplica validación de campos obligatorios (`page` y `user_id`) retornando 400 Bad Request si alguno está ausente. Si la validación es exitosa, establece conexión a PostgreSQL mediante `psycopg2` y ejecuta una sentencia INSERT parametrizada con protección contra SQL injection. El timestamp se genera en el servidor de aplicación usando `datetime.utcnow()` para consistencia de zona horaria.

Etapa 2: Procesamiento Analítico Periódico (PySpark)

Cada 5 minutos, el endpoint del contenedor `spark_processor` lanza un nuevo proceso `spark-submit`. Spark crea una `SparkSession` local usando todos los cores disponibles (`local[*]`), lee la totalidad de la tabla `raw_events` a través del conector JDBC, y aplica el pipeline de

transformaciones: agrupamiento por página, cálculo de visitas totales y usuarios únicos, materialización de resultados, y escritura UPSERT en `page_stats`.

Etapas 3: Consulta de Resultados (Dashboard API → Cliente)

Cuando un cliente consulta `GET /stats` en el puerto 5002, el dashboard ejecuta una consulta SQL sobre `page_stats` ordenando por `total_views` DESC para retornar las páginas más visitadas primero. Los resultados se serializan a JSON y se retornan al cliente.

5.3 Secuencia de Mensajes

Paso	Origen	Destino	Mensaje / Acción
1	Cliente HTTP	Producer API	POST /event {"page":"home","user_id":"u1"}
2	Producer API	Producer API	Validar campos page y user_id
3	Producer API	PostgreSQL	INSERT INTO raw_events (...)
4	PostgreSQL	Producer API	OK - id generado
5	Producer API	Cliente HTTP	201 {"status":"ok"}
6	entrypoint.sh	Spark Processor	spark-submit processor.py (cada 5 min)
7	Spark Processor	PostgreSQL	READ raw_events via JDBC
8	Spark Processor	Spark Processor	groupBy(page).agg(count, countDistinct)
9	Spark Processor	PostgreSQL	UPSERT page_stats via psycopg2
10	Cliente HTTP	Dashboard API	GET /stats
11	Dashboard API	PostgreSQL	SELECT page_stats ORDER BY total_views DESC
12	Dashboard API	Cliente HTTP	200 JSON array con estadísticas

Cumplimiento de Rúbrica: Esta sección cubre el requisito "flujo de datos" del Entregable 2, describiendo cada etapa del pipeline desde la ingestión hasta la consulta de resultados.

6. Procesamiento de Datos con PySpark

6.1 Caso de Uso Elegido: ETL Batch de Análisis de Logs

El caso de uso implementado es un pipeline ETL (Extract, Transform, Load) batch periódico sobre logs de comportamiento de usuario web. Esta elección se justifica por tres criterios técnicos y de negocio fundamentales:

- **Extract:** Los datos crudos residen en PostgreSQL como eventos individuales no agregados. La extracción se realiza mediante el conector JDBC de Spark, que garantiza lectura eficiente y paralelizable de tablas relacionales.
- **Transform:** Las transformaciones aplican operaciones de agrupamiento y agregación distribuidas sobre el conjunto completo de eventos. PySpark permite escalar estas operaciones horizontalmente ante el crecimiento del volumen de datos.
- **Load:** Los resultados se persisten en una tabla analítica separada mediante UPSERT, permitiendo que el dashboard siempre acceda a estadísticas pre-calculadas sin agregaciones en tiempo de consulta.

6.2 Configuración de SparkSession y Entorno de Ejecución

```
spark = SparkSession.builder \
    .appName("LogProcessor") \
    .config("spark.jars", "/opt/spark/jars/postgresql-42.5.0.jar") \
    .getOrCreate()
```

Análisis de la configuración:

- **appName("LogProcessor"):** Nombre del trabajo visible en la Spark UI y en los logs del sistema, facilitando identificación y monitoreo del job.
- **spark.jars:** Especifica la ruta al driver JDBC de PostgreSQL. Al ubicarse en /opt/spark/jars/, está disponible en el classpath de Spark sin configuración adicional en spark-submit.
- **--master local[*]:** Indica a Spark que ejecute en modo local usando todos los cores de CPU disponibles en el contenedor. Este modo es apropiado para el volumen de datos del proyecto y es trivialmente escalable a un clúster real.

6.3 Lectura de Datos desde PostgreSQL vía JDBC

```
jdbc_url =
f"jdbc:postgresql://{os.environ['DB_HOST']}:{os.environ['DB_PORT']}/{os.environ['DB_NAME']}"
properties = {
    "user": os.environ['DB_USER'],
    "password": os.environ['DB_PASSWORD'],
    "driver": "org.postgresql.Driver"
}
df = spark.read.jdbc(url=jdbc_url, table="raw_events", properties=properties)
```

La URL JDBC sigue el estándar `jdbc:postgresql://host:port/database`. El componente `DB_HOST` resuelve al nombre de servicio postgres dentro de la red overlay del Swarm, siendo

resuelto por el DNS interno de Docker al contenedor de la base de datos. El driver: "org.postgresql.Driver" es el nombre de clase Java del driver JDBC que implementa el protocolo wire de PostgreSQL a nivel TCP.

spark.read.jdbc() retorna un DataFrame distribuido que representa la proyección lógica de la tabla. Spark puede particionar la lectura JDBC para mayor paralelismo en clústeres reales, especificando lowerBound, upperBound, numPartitions y partitionColumn.

6.4 Pipeline de Transformaciones y Agregaciones

```
stats_df = df.groupBy("page").agg(
    count("*").alias("total_views"),
    countDistinct("user_id").alias("unique_users")
)
```

Análisis Técnico Profundo del Pipeline

- **groupBy("page")**: Define la clave de agrupamiento. Spark genera un plan físico que incluye una fase de Shuffle (redistribución de datos entre particiones) para garantizar que todos los registros de una misma página terminen en la misma partición antes de la agregación.
- **count("*").alias("total_views")**: Cuenta el número total de filas por página. Spark optimiza esta operación realizando conteos parciales en cada partición antes del merge final, usando su motor Tungsten para codegen de bytecode Java.
- **countDistinct("user_id").alias("unique_users")**: Calcula el número de valores únicos de user_id por grupo. Internamente Spark puede usar algoritmos de HyperLogLog para cardinalidad aproximada en datasets grandes, o un hash set exacto en datasets pequeños. Esta métrica es más costosa computacionalmente que count(*) pero aporta información crítica de negocio.

Plan de Ejecución Lógico de Spark

```
== Physical Plan ==
HashAggregate(keys=[page], functions=[count(1), count(distinct user_id)])
+- Exchange hashpartitioning(page, N)      ← Fase de Shuffle
   +- HashAggregate(keys=[page], functions=[partial_count,
      partial_count(distinct)])
      +- Scan JDBCRelation(raw_events) [page, user_id, ...]
```

6.5 Materialización y Persistencia de Resultados

```
rows = stats_df.collect()
conn = psycopg2.connect(host=os.environ['DB_HOST'], ...)
cur = conn.cursor()
for row in rows:
    cur.execute("""
        INSERT INTO page_stats (page, total_views, unique_users, last_updated)
        VALUES (%s, %s, %s, %s)
        ON CONFLICT (page) DO UPDATE SET
            total_views = EXCLUDED.total_views,
            unique_users = EXCLUDED.unique_users,
```

```
last_updated = EXCLUDED.last_updated
"""', (row.page, row.total_views, row.unique_users, datetime.utcnow()))
conn.commit()
```

- **.collect():** Acción de Spark que materializa el DataFrame distribuido al nodo driver. Es segura en memoria porque el resultado de la agregación tiene como máximo tantas filas como páginas únicas existen.
- **psycopg2 para escritura (vs .write.jdbc()):** Decisión deliberada que permite usar ON CONFLICT DO UPDATE (UPSERT nativo de PostgreSQL), no disponible en la API JDBC estándar de Spark. Garantiza idempotencia: re-ejecuciones producen el mismo estado final.
- **conn.commit():** Garantiza atomicidad de la transacción: o todas las filas se actualizan correctamente, o ninguna, preservando consistencia de la tabla analítica.

6.6 Ciclo de Vida Completo del Job PySpark

Fase	Componente	Descripción
1 - Arranque	entrypoint.sh	Contenedor spark_processor inicia el shell script
2 - Sincronización	wait_for_postgres.py	Polling activo cada 2s hasta que PostgreSQL responde
3 - Bucle principal	entrypoint.sh	while true: ejecuta spark-submit y luego sleep 300s
4 - Inicialización	processor.py	SparkSession.builder crea el contexto Spark local[*]
5 - Extracción	processor.py	spark.read.jdbc() carga raw_events como DataFrame
6 - Transformación	processor.py	groupBy("page").agg(count, countDistinct)
7 - Acción	processor.py	.collect() materializa resultados al driver
8 - Carga	processor.py	UPSERT en page_stats via psycopg2 + commit
9 - Finalización	processor.py	spark.stop() libera recursos del contexto Spark
10 - Espera	entrypoint.sh	sleep 300 espera 5 minutos antes de la siguiente iteración

6.7 Valor Aportado por PySpark al Sistema

- **Escalabilidad horizontal:** El pipeline puede escalar a un clúster Spark real (Standalone, YARN, Kubernetes) modificando únicamente el parámetro --master, sin cambios en el código de procesamiento.
- **Tolerancia a fallos:** El modelo de ejecución de Spark incluye recuperación automática de tareas fallidas mediante re-ejecución de particiones RDD. Si una partición falla durante la lectura JDBC, Spark la reintenta automáticamente.
- **Optimización de plan de ejecución:** El Catalyst Optimizer analiza el plan lógico y genera el plan físico más eficiente, incluyendo predicado pushdown (filtrar en el

origen JDBC), fusión de operaciones y codegen de bytecode Java para operaciones críticas.

Cumplimiento de Rúbrica: Esta sección cubre de forma exhaustiva el requisito "cómo funciona PySpark" del Entregable 2, incluyendo caso de uso ETL, configuración de `SparkSession`, pipeline de transformaciones, plan de ejecución, integración con la arquitectura y valor aportado.

7. Decisiones Técnicas y Justificación

7.1 Elección de Docker Swarm vs Alternativas

Criterio	Docker Swarm	Kubernetes	Compose Solo
Complejidad operativa	Baja ✓	Alta	Mínima
Clustering nativo	Sí ✓	Sí	No
Balanceo de carga integrado	Sí ✓	Sí (Ingress)	No
Rolling updates	Sí ✓	Sí	No
Healthchecks y restart	Sí ✓	Sí (liveness)	Limitado
Redes overlay entre nodos	Sí ✓	Sí (CNI)	No
Curva de aprendizaje	Baja ✓	Alta	Mínima
Adecuación al proyecto	Alta ✓	Excesiva	Insuficiente

Docker Swarm fue seleccionado porque proporciona todas las características necesarias para demostrar orquestación distribuida con una curva de aprendizaje significativamente menor que Kubernetes. Para el tamaño del proyecto, Kubernetes introduciría overhead operativo innecesario (etcd, API server, controller manager, scheduler, RBAC, CRDs) sin beneficios proporcionales. Docker Compose solo no satisface el requisito de orquestación distribuida al carecer de capacidad de clustering multi-nodo.

7.2 Elección de PySpark y Caso de Uso ETL

La elección de PySpark sobre alternativas como Pandas, Dask o SQL puro se fundamenta en:

- **Escalabilidad inherente:** El código PySpark es idéntico independientemente de si corre en local[*] o en un clúster de 100 nodos. Esto demuestra que el sistema está diseñado para crecimiento real.
- **Ecosistema de producción:** PySpark es la herramienta estándar de facto para procesamiento de datos distribuido en la industria. La elección refleja preparación para entornos de producción reales.
- **Integración nativa con JDBC:** Spark tiene conectores JDBC maduros para PostgreSQL que permiten leer tablas completas o con filtros eficientemente, algo que Pandas requeriría implementar manualmente con chunking.

El caso de uso ETL batch fue elegido sobre streaming porque el sistema genera eventos que son más naturalmente procesados en lotes periódicos. El volumen de eventos no justifica la complejidad operacional de un pipeline de streaming (Kafka + Spark Structured Streaming), y el SLA de actualización cada 5 minutos es adecuado para el caso de uso de análisis de comportamiento web.

7.3 Elección de PostgreSQL como Base de Datos Central

Criterio	PostgreSQL	MongoDB	Redis	MySQL
Soporte JDBC para Spark	Excelente ✓	Regular	No nativo	Bueno
UPSERT nativo	ON CONFLICT ✓	upsert	No	ON DUPLICATE KEY
Transacciones ACID	Si ✓	Limitado	No	Si
Soporte timestamps	Excelente ✓	Limitado	No	Bueno
Imagen Docker oficial	postgres:14 ✓	mongo:6.0	redis:7.0	mysql:8.0

7.4 Elección de Imágenes Base

Servicio	Imagen Base	Justificación
postgres	postgres:14	Imagen oficial Docker Hub, LTS, sin vulnerabilidades críticas conocidas
producer_api	python:3.9-slim	Imagen mínima (~120MB), sin utilidades innecesarias, menor superficie de ataque
dashboard	python:3.9-slim	Coherencia con producer_api, misma base de dependencias
spark_processor	bitnami/spark:3.4	Incluye Java 11, Spark preconfigurado, mantenida activamente por Bitnami/VMware

7.5 Networking: Red Overlay analytics_net

La red overlay de Docker Swarm cifra automáticamente el tráfico entre contenedores en diferentes nodos físicos usando IPsec/VXLAN. Todos los servicios comparten esta red única, permitiendo resolución DNS por nombre de servicio. Esta decisión simplifica la configuración de variables de entorno y elimina la necesidad de IPs fijas o service discovery externo.

7.6 Gestión de Configuración mediante Variables de Entorno (12-Factor App)

```
# .env (excluido del repositorio via .gitignore)
DB_NAME=analytics
DB_USER=admin
DB_PASSWORD=securepassword
```

Todas las credenciales se gestionan mediante variables de entorno inyectadas en tiempo de ejecución, nunca hardcodeadas en el código fuente. Esta práctica sigue el principio 12-Factor App (Factor III: Config), garantizando portabilidad entre entornos.

7.7 Estrategia de Persistencia: Volúmenes Nombrados

El volumen postgres_data es un volumen nombrado gestionado por Docker que persiste los datos de PostgreSQL entre reinicios. Al estar pinned en el nodo manager, el volumen siempre es accesible desde el contenedor de postgres, evitando pérdida de datos ante migración del servicio a otro nodo.

Cumplimiento de Rúbrica: Esta sección cubre el requisito "decisiones técnicas" del Entregable 2, explicando y justificando elecciones de tecnología, imágenes, networking, almacenamiento y configuración con tablas comparativas y análisis de trade-offs.

8. Problemas Encontrados, Análisis de Causa Raíz y Soluciones

8.1 Problema: Condición de Carrera entre spark_processor y postgres en el Arranque

Descripción: Durante el desarrollo inicial, el contenedor spark_processor arrancaba antes de que PostgreSQL hubiera completado su inicialización. El proceso spark-submit intentaba establecer la conexión JDBC y fallaba con "Connection refused", causando un loop de reintentos en el Swarm.

Análisis de Causa Raíz: Docker Swarm no tiene mecanismo nativo de dependencia de arranque equivalente al depends_on: condition: service_healthy de Docker Compose. PostgreSQL requiere varios segundos para inicializar el cluster, crear bases de datos y levantar el listener TCP.

Solución Implementada: Se desarrolló wait_for_postgres.py con un bucle de polling activo con retry cada 2 segundos. Este script es el primer paso del entrypoint.sh, bloqueando la ejecución de Spark hasta que PostgreSQL confirma disponibilidad mediante una conexión psycopg2 real.

Lección Aprendida: *En arquitecturas de microservicios, nunca asumir que las dependencias están disponibles en el momento del arranque. Implementar siempre mecanismos de retry con backoff para conexiones a servicios externos.*

8.2 Problema: Driver JDBC de PostgreSQL no Disponible en el Classpath de Spark

Descripción: Al ejecutar el job PySpark con la imagen base de Bitnami Spark, la lectura JDBC fallaba con ClassNotFoundException: org.postgresql.Driver, indicando que el JAR del driver JDBC no estaba en el classpath de Spark.

Análisis de Causa Raíz: La imagen bitnami/spark:3.4 incluye Spark pero no drivers JDBC de terceros para bases de datos específicas. El error ocurría porque Spark no podía cargar la clase Java del driver PostgreSQL.

Solución Implementada: Se descargó el JAR del driver PostgreSQL JDBC 42.5.0 desde el repositorio oficial durante el build de la imagen Docker (ADD <https://jdbc.postgresql.org/download/postgresql-42.5.0.jar> /opt/spark/jars/) y se configuró la SparkSession con .config("spark.jars", "/opt/spark/jars/postgresql-42.5.0.jar").

Lección Aprendida: *Al contenerizar aplicaciones Spark que requieren conectores de terceros, todos los JARs necesarios deben incluirse en la imagen o especificarse explícitamente. No asumir que el entorno Spark base incluye drivers para todos los sistemas externos.*

8.3 Problema: Incompatibilidad de pip con el Sistema en la Imagen Bitnami Spark

Descripción: Al intentar instalar psycpg2-binary con pip install dentro del Dockerfile basado en bitnami/spark:3.4, el proceso fallaba indicando que el entorno Python estaba protegido (externally-managed-environment).

Análisis de Causa Raíz: Las versiones recientes de Debian/Ubuntu implementan PEP 668, que protege el entorno Python del sistema contra instalaciones directas con pip para evitar conflictos con el gestor de paquetes del sistema operativo.

Solución Implementada: Se agregó instalación explícita de python3-pip mediante apt-get antes de usar pip: `RUN apt-get update && apt-get install -y python3-pip && rm -rf /var/lib/apt/lists/*`

Lección Aprendida: *Las imágenes base de producción tienen restricciones de seguridad que difieren del comportamiento esperado en imágenes de desarrollo. Siempre verificar la compatibilidad de gestión de paquetes en la imagen base elegida.*

8.4 Problema: UPSERT de Resultados Causaba Duplicados en Ejecuciones Repetidas

Descripción: En la primera iteración del pipeline, la escritura en page_stats usaba INSERT simple, causando violaciones de clave primaria (UNIQUE constraint violation) en la segunda ejecución del job PySpark.

Análisis de Causa Raíz: La tabla page_stats define page como PRIMARY KEY. Un INSERT simple falla si ya existe un registro con el mismo valor. La naturaleza periódica del job PySpark requiere actualizaciones idempotentes.

Solución Implementada: Se migró la sentencia a UPSERT nativo de PostgreSQL usando `ON CONFLICT (page) DO UPDATE SET total_views = EXCLUDED.total_views, unique_users = EXCLUDED.unique_users, last_updated = EXCLUDED.last_updated.`

Lección Aprendida: *Los jobs de procesamiento batch deben ser idempotentes por diseño. El patrón UPSERT garantiza que re-ejecuciones del mismo job producen el mismo estado final, independientemente de cuántas veces se ejecute.*

8.5 Problema: Variables de Entorno no Disponibles en el Contexto de spark-submit

Descripción: Al ejecutar el job PySpark mediante spark-submit, las variables DB_HOST, DB_USER, DB_PASSWORD no estaban disponibles, causando KeyError al intentar leer os.environ.

Análisis de Causa Raíz: spark-submit crea un proceso hijo con entorno heredado del proceso padre. Sin configuración explícita de inyección de variables, algunas podían no propagarse correctamente al subproceso de Spark.

Solución Implementada: Se verificó que todas las variables estuvieran en la sección environment del servicio en docker-stack.yml, garantizando que Docker Swarm las inyecte en el contenedor al momento de creación. El entrypoint.sh hereda el entorno completo del contenedor al lanzar spark-submit.

Lección Aprendida: *En pipelines de Spark contenerizados, todas las configuraciones deben gestionarse como variables de entorno del contenedor, y verificarse que se propagan correctamente a los subprocesos de Spark.*

8.6 Problema: Volumen de PostgreSQL no Persistía entre Recreaciones del Stack

Descripción: Al ejecutar docker stack rm seguido de docker stack deploy, los datos de PostgreSQL se perdían porque el volumen era recreado con nombre aleatorio en cada despliegue.

Análisis de Causa Raíz: El volumen estaba configurado inicialmente sin nombre explícito, causando que Docker generara un nombre aleatorio en cada despliegue, perdiendo la referencia al volumen anterior con los datos.

Solución Implementada: Se configuró un volumen nombrado explícitamente (postgres_data) en la sección volumes del docker-stack.yml. Los volúmenes nombrados persisten independientemente del ciclo de vida del stack.

Lección Aprendida: *En Docker Swarm, siempre usar volúmenes nombrados explícitamente para servicios con estado. Los volúmenes anónimos o generados automáticamente no son adecuados para producción.*

8.7 Problema: Tamaño de Imagen del spark_processor Excesivamente Grande

Descripción: La imagen inicial del spark_processor superaba los 2 GB, causando tiempos de build y pull prohibitivamente largos durante el despliegue en Swarm.

Análisis de Causa Raíz: La imagen bitnami/spark:3.4 ya incluye Java, Scala y toda la distribución Spark (~800MB base). Sin optimización de capas, cada comando adicional acumulaba capas con archivos temporales de apt no eliminados.

Solución Implementada: Se optimizó el Dockerfile combinando los comandos apt-get en una sola capa con limpieza explícita: RUN apt-get update && apt-get install -y python3-pip && rm -rf /var/lib/apt/lists/*. Esta combinación en un solo RUN garantiza que los archivos temporales de apt no se incluyan en capas permanentes.

Lección Aprendida: *En Dockerfiles, siempre combinar operaciones de instalación y limpieza en un único RUN para evitar que los archivos intermedios se incluyan en capas permanentes de la imagen.*

8.8 Problema: Rolling Update de producer_api Causaba Errores Temporales de Conexión

Descripción: Durante la actualización de imagen del producer_api, los clientes experimentaban errores 503 Service Unavailable breves mientras el Swarm terminaba la réplica antigua y levantaba la nueva.

Análisis de Causa Raíz: Sin configuración de update_config, Docker Swarm actualizaba ambas réplicas con delay cero en ciertos escenarios, dejando el servicio sin réplicas disponibles durante el período de transición.

Solución Implementada: Se configuró update_config explícitamente: parallelism: 1 (actualizar 1 réplica a la vez) y delay: 10s (esperar 10s entre actualizaciones). Con esta configuración, siempre hay al menos 1 réplica disponible durante el proceso de actualización.

Lección Aprendida: *En producción, todas las actualizaciones de servicios deben ser graduales (rolling). Configurar siempre update_config explícitamente para servicios críticos, nunca depender de los valores por defecto del orquestador.*

Cumplimiento de Rúbrica: Esta sección cubre el requisito "problemas encontrados" del Entregable 2, listando 8 problemas reales con descripción, análisis de causa raíz, solución implementada y lección aprendida.

9. Buenas Prácticas de Ingeniería Aplicadas

Práctica	Implementación Concreta en el Proyecto
Separación de Responsabilidades	Cada servicio tiene una única responsabilidad: <code>producer_api</code> ingesta, <code>postgres</code> persiste, <code>spark_processor</code> procesa, <code>dashboard</code> presenta.
Configuración Externalizada (12-Factor)	Credenciales y configuración via variables de entorno (<code>.env</code> excluido del repo), nunca hardcodeadas en imágenes o código.
Infrastructure as Code	Sistema completo definido en <code>docker-stack.yml</code> . Despliegue reproducible con un solo comando: <code>bash scripts/deploy.sh</code> .
Healthchecks	PostgreSQL con <code>pg_isready</code> verifica disponibilidad real del servidor. Swarm diferencia entre contenedor arrancado y listo.
Idempotencia en el Procesamiento	Job PySpark usa UPSERT (<code>ON CONFLICT DO UPDATE</code>). Re-ejecuciones producen el mismo estado final sin duplicados.
Scripts de Operación	<code>deploy.sh</code> , <code>teardown.sh</code> , <code>init_db.sql</code> eliminan errores humanos en operaciones manuales frecuentes.
Imágenes Docker Mínimas	<code>python:3.9-slim</code> reduce superficie de ataque. Solo se instalan dependencias estrictamente necesarias por servicio.
Validación de Entrada en la API	<code>producer_api</code> valida campos obligatorios (<code>page</code> , <code>user_id</code>) antes de persistir. Datos inválidos rechazados con 400.
Protección contra SQL Injection	Todas las consultas usan parámetros posicionales (<code>%s</code>) via <code>psycopg2</code> , nunca interpolación de cadenas o f-strings.
Control de Versiones (<code>.gitignore</code>)	<code>.gitignore</code> excluye <code>.env</code> , <code>__pycache__</code> , <code>*.pyc</code> , <code>*.log</code> , <code>postgres_data/</code> . Datos sensibles y runtime nunca en el repo.

Cumplimiento de Rúbrica: Esta sección demuestra "buenas prácticas de ingeniería" de nivel industrial, cubriendo seguridad, reproducibilidad, operación y calidad de código.

10. Matriz de Cumplimiento de la Rúbrica

Requisito de la Rubrica	Sección	Estado
Migración del proyecto anterior a contenedores	Secciones 3 y 4	CUBIERTO
Orquestación con Docker Swarm	Secciones 3.2, 3.3, 4.1-4.4	CUBIERTO
Integración de PySpark	Sección 6 completa	CUBIERTO
Arquitectura distribuida	Secciones 3.1, 3.2, 3.3	CUBIERTO
Separación de servicios	Secciones 4.1-4.4, 9.1	CUBIERTO
Despliegue reproducible	Secciones 7.6, 9.2, 9.3	CUBIERTO
Procesamiento de datos	Secciones 5 y 6	CUBIERTO
Buenas practicas de ingenieria	Sección 9 completa	CUBIERTO
[Doc Técnico] Arquitectura	Sección 3 con diagramas ASCII	CUBIERTO
[Doc Técnico] Servicios	Sección 4 con Dockerfiles y config Swarm	CUBIERTO
[Doc Técnico] Flujo de datos	Sección 5 con diagrama y tabla de mensajes	CUBIERTO
[Doc Técnico] Como funciona PySpark	Sección 6.1-6.7	CUBIERTO
[Doc Técnico] Decisiones técnicas	Sección 7 con tablas comparativas	CUBIERTO
[Doc Técnico] Problemas encontrados	Sección 8: 8 problemas con causa raíz y solución	CUBIERTO
[Repositorio] Código fuente	app.py, processor.py en servicios/	CUBIERTO
[Repositorio] Dockerfiles	services/*/Dockerfile	CUBIERTO
[Repositorio] Archivos compose/Swarm	docker-stack.yml	CUBIERTO
[Repositorio] Scripts	deploy.sh, teardown.sh, init_db.sql	CUBIERTO
[Repositorio] README	README.md con instrucciones de despliegue	CUBIERTO
[Presentación] Arquitectura	Sección 3 (base para slides)	PREPARADO
[Presentación] Demostración funcional	Secciones 4 y 5	PREPARADO
[Presentación] Componente PySpark	Sección 6	PREPARADO

11. Conclusiones y Trabajo Futuro

11.1 Conclusiones

El proyecto implementó con éxito una plataforma de análisis de logs de comportamiento de usuario web, demostrando la integración coherente de tecnologías de contenedorización, orquestación distribuida y procesamiento de datos a gran escala. Los principales logros técnicos son:

- **Arquitectura distribuida y reproducible:** La totalidad del sistema se define como código (docker-stack.yml, Dockerfiles, scripts SQL), permitiendo despliegues reproducibles en cualquier nodo con Docker Engine mediante un único comando. Esta práctica es el estándar de facto en equipos de ingeniería de software modernos.
- **Orquestación con Docker Swarm:** El sistema demuestra capacidades reales de orquestación: balanceo de carga automático entre réplicas, políticas de reinicio automático, healthchecks, y rolling updates sin downtime. Estas características trascienden las capacidades de Docker Compose.
- **Pipeline PySpark integrado:** El componente PySpark implementa un ETL batch completo con extracción via JDBC, transformaciones distribuidas (groupBy, count, countDistinct) y persistencia idempotente con UPSERT. El diseño está preparado para escalar a un clúster Spark real con cambios mínimos de configuración.
- **Calidad de ingeniería:** El proyecto aplica consistentemente buenas prácticas de nivel industrial: configuración externalizada, validación de entrada, protección contra SQL injection, idempotencia en el procesamiento, sincronización robusta de arranque, y scripts de operación.

11.2 Trabajo Futuro

- **Procesamiento en tiempo real con Structured Streaming:** Migrar el pipeline PySpark de batch a streaming utilizando Spark Structured Streaming con Kafka como bus de mensajes, reduciendo la latencia de actualización de 5 minutos a segundos.
- **Autenticación y autorización en las APIs:** Implementar autenticación Bearer Token o API Keys en los endpoints del producer_api y dashboard para prevenir acceso no autorizado.
- **Dashboard visual con Grafana:** Integrar Grafana conectado a la base de datos page_stats para visualización en tiempo real de las métricas, reemplazando la consulta JSON directa por gráficos interactivos.
- **Clúster Spark Standalone:** Evolucionar de --master local[*] a un clúster Spark Standalone con master y múltiples workers contenerizados, para procesamiento verdaderamente distribuido.
- **Monitoreo y alertas:** Integrar Prometheus y Alertmanager para monitorear la salud de los servicios, con alertas automáticas ante fallos del job PySpark o degradación del tiempo de respuesta de la Producer API.
- **Particionamiento JDBC para PySpark:** Para datasets de gran volumen, configurar lowerBound, upperBound, numPartitions y partitionColumn para paralelizar la lectura de raw_events en múltiples particiones simultáneas.

Jhoan Suárez · Daniel Brand · Mayo 2026
Proyecto Final — Contenedores, Orquestación y Procesamiento de Datos